PROJECT DOCUMENTATION

OptiCrop Smart Agricultural Production Optimization

A data-driven crop recommendation engine integrating soil nutrient levels, climate parameters, and machine learning to optimize agricultural production across diverse agro-climatic zones.

Team Lead: Peddehapu Niharika
Members: Polupalli Sunil | Romeo Mhakayakora | Sk Salomi | Jotshna Aditya Tadi
Version 1.0 | July 2026

Table of Contents

1. Executive Summary and Problem Domain

Agriculture remains the backbone of economies across developing nations, yet the sector continues to face systemic inefficiencies in crop selection and resource allocation. Farmers, particularly those operating small to medium-sized holdings in semi-urban and rural landscapes, routinely rely on generational knowledge or anecdotal advice when deciding which crop to cultivate on a given parcel of land. This heuristic-driven approach, while rooted in tradition, fails to account for the precise chemical composition of the soil, the micro-climatic conditions unique to each growing season, and the increasingly volatile patterns of rainfall and temperature driven by broader climate shifts. The consequences are significant: sub-optimal crop yields, wasteful expenditure on fertilizers that do not match the soil's actual nutrient profile, and long-term degradation of arable land through imbalanced nutrient replenishment.

OptiCrop addresses these challenges by providing a data-driven crop recommendation engine that synthesizes seven critical agricultural input parameters — Nitrogen (N), Phosphorous (P), and Potassium (K) levels in the soil, soil temperature, atmospheric humidity, soil pH, and seasonal rainfall — and maps them against a trained machine learning model to predict the most suitable crop for a given plot. The system is designed to serve three distinct user archetypes: individual farmers who need a simple, offline-capable tool for real-time field decisions; agricultural researchers and data scientists who require cloud-hosted dashboards for regional analysis and longitudinal trend monitoring; and policymakers at government agencies and NGOs who benefit from aggregated, queryable datasets that inform subsidy allocation, seed distribution programs, and climate-adaptation strategies.

The problem domain spans soil chemistry, climatology, and predictive analytics. Soil nutrients N, P, and K are the primary macronutrients governing plant growth — nitrogen drives foliage and vegetative development, phosphorous supports root establishment and energy transfer through ATP, and potassium regulates water uptake, enzyme activation, and disease resistance. When any of these elements fall outside the optimal range for a specific crop, the result is stunted growth, reduced yield, or complete crop failure. Similarly, soil pH determines nutrient bioavailability: most crops thrive in a slightly acidic to neutral range (pH 6.0–7.5), while highly acidic or alkaline soils lock essential nutrients into insoluble compounds that plants cannot absorb. Temperature and humidity influence both the metabolic rate of the plant and the prevalence of pests and fungal pathogens, while rainfall — both its total volume and its temporal distribution across the growing season — determines whether a crop receives adequate water without suffering waterlogging or drought stress.

OptiCrop integrates all of these variables into a unified prediction pipeline. By training on a curated agricultural dataset that maps historical soil-climate-crop combinations to their actual yield outcomes, the system learns the multi-dimensional decision boundaries that separate one optimal crop from another. The end result is a recommendation that is not merely statistical but agronomically meaningful — a prediction that a farmer can act upon with confidence, knowing that it reflects the collective intelligence of thousands of prior growing seasons encoded into a robust machine learning model.

2. Scenarios

The following scenarios illustrate how OptiCrop serves its three primary user archetypes across distinct operational contexts. Each scenario is grounded in realistic agricultural settings in India and demonstrates the system's ability to translate complex environmental data into actionable crop recommendations.

Scenario 1: Smart Crop Recommendation for Farmers

A farmer in a semi-arid region of Andhra Pradesh prepares for the upcoming kharif season. Over the past several years, she has noticed declining yields from her primary crop of paddy rice and suspects that the soil on her three-acre plot has become nutrient-depleted. Rather than investing in an expensive soil testing consultation that would require shipping samples to a distant laboratory and waiting weeks for results, she opens the OptiCrop application on her Android smartphone — a device she has owned for two years and which she uses primarily for voice calls and messaging. The application loads instantly because it was designed for offline operation; the machine learning model is embedded directly within the APK package, and no internet connection is required.

On the home screen, she is presented with a clean, card-based input form labeled "Enter Your Soil & Climate Data." Each input field is accompanied by a contextual tooltip: the Nitrogen field explains that values typically range from 0 to 140 kg/ha, the Phosphorous field notes a range of 5 to 145 kg/ha, and so on. She enters the values from her most recent soil test report — Nitrogen at 42, Phosphorous at 58, Potassium at 35, pH at 6.3, temperature at 28 degrees Celsius, humidity at 65 percent, and rainfall at 180 mm. She taps the "Get Recommendation" button at the bottom of the form.

Within milliseconds, the application deserializes the pre-loaded crop_recommendation_model.pkl file using the joblib library, constructs a feature vector from her seven input values, passes it through the trained ensemble model, and returns a ranked list of the top three recommended crops. The first recommendation — "Maize" — is displayed prominently in a green success banner with a confidence score, followed by "Soybean" and "Chickpea" as alternatives. Each recommendation card includes a brief agronomic rationale: "Maize is well-suited to your moderate nitrogen levels, slightly acidic pH, and the prevailing rainfall pattern." The farmer can tap any card to see a more detailed breakdown of how her input values compare to the ideal ranges for that crop.

This scenario demonstrates the core value proposition of OptiCrop: transforming complex agronomic data into an actionable, field-ready recommendation in under five seconds, without requiring internet connectivity, specialized knowledge, or additional hardware.

Scenario 2: Crop Suitability and Environmental Assessment

An agricultural extension officer visits a cooperative farm in Maharashtra where a group of smallholder farmers is considering transitioning from traditional jowar (sorghum) cultivation to cotton, driven by recent market price increases. The officer needs to assess whether the cooperative's 50-hectare plot is agronomically compatible with cotton's specific soil and climate requirements. Rather than relying on general guidelines from agricultural handbooks — which may not account for the plot's unique micro-conditions — the officer uses OptiCrop's Crop Suitability Assessment mode.

In this mode, the officer first selects "Cotton" from a dropdown menu of supported crops, which triggers the application to display the ideal parameter ranges for cotton: Nitrogen between 30–60 kg/ha, Phosphorous between 40–80 kg/ha, Potassium between 25–50 kg/ha, pH between 6.0–7.0, temperature between 25–35 degrees Celsius, humidity between 50–80 percent, and rainfall between 500–1200 mm. The officer then enters the actual soil and climate readings collected from the cooperative's field sensors and local weather station records. The application performs a point-by-point comparison, generating a compatibility score for each parameter.

The resulting assessment reveals that while the plot's nitrogen and phosphorous levels fall comfortably within cotton's preferred range, the potassium level is marginally deficient at 18 kg/ha (below the recommended minimum of 25 kg/ha), and the average rainfall of 420 mm falls short of the 500 mm threshold. The application highlights these discrepancies in an amber warning panel and recommends targeted interventions: applying muriate of potash to raise potassium levels and implementing drip irrigation to supplement rainfall during critical growth stages. The officer shares this assessment with the cooperative members, enabling them to make an informed decision about the crop transition rather than proceeding blindly based on market signals alone.

This scenario illustrates how OptiCrop functions not only as a predictive tool but also as a diagnostic and educational resource that bridges the gap between raw environmental data and practical agronomic decision-making.

Scenario 3: Agricultural Research and Policy Planning

A research team at a state-level agricultural university is conducting a longitudinal study on the impact of changing monsoon patterns on crop suitability across five districts in Karnataka. The team needs access to a centralized, queryable repository of soil-climate-crop predictions that can be aggregated by district, season, and year to identify emerging trends — such as a gradual shift in the optimal crop profile for a district that historically favored rice but may be transitioning toward drought-resistant millets.

The researchers access the cloud-hosted version of OptiCrop, which is deployed on Render (with AWS as the cloud database backend). Unlike the offline Android APK, the cloud version logs every prediction query — including the input parameters, the predicted crop, the confidence score, and the user's district identifier — into a persistent SQLite database hosted on the server. Over time, this database accumulates a rich dataset of real-world agricultural queries that can be analyzed using standard SQL queries or exported to CSV for further statistical modeling in tools like R or Python's pandas library.

The research team constructs a dashboard that visualizes the frequency distribution of recommended crops by district over the past three monsoon seasons. They observe a statistically significant increase in the recommendation frequency for "Ragi" (finger millet) and "Groundnut" in the northern districts, correlating with declining average rainfall readings submitted by users in those regions. This data forms the empirical foundation for a policy brief submitted to the state Department of Agriculture, recommending a reallocation of seed subsidy budgets toward drought-resistant crop varieties for the affected districts. Without OptiCrop's centralized logging infrastructure, assembling this dataset would have required months of manual field surveys at considerably greater expense.

This scenario underscores OptiCrop's dual utility: while it serves individual farmers at the point of need, it simultaneously generates institutional-grade data that supports evidence-based agricultural policy at regional and national scales.

3. Technical Architecture and System Flow

OptiCrop is designed as a classical three-tier architecture that cleanly separates concerns across the presentation, application logic, and data/model layers. This separation ensures that each layer can be developed, tested, and deployed independently, and that changes to one layer — such as swapping the machine learning model or migrating the database — do not cascade into breaking changes across the entire system.

Presentation Layer (Frontend)

The presentation layer is implemented as a single-page web application served through Flask's template engine. The primary interface file, index.html, is structured as a series of responsive input cards — one for each of the seven agricultural parameters — arranged in a grid layout that adapts to both desktop and mobile viewports. Each card contains a labeled input field, a range indicator showing the scientifically accepted bounds for that parameter, and a tooltip icon that provides contextual guidance (for example, explaining that soil pH is measured on a logarithmic scale from 0 to 14, with 7 being neutral).

When the user completes the form and clicks the "Get Recommendation" button, the frontend assembles the seven input values into a JSON payload and initiates an asynchronous fetch request to the Flask backend's /predict endpoint. This asynchronous approach ensures that the UI remains responsive during the prediction computation — the user sees a loading spinner or progress indicator rather than a frozen screen — and allows for graceful error handling if the server is unreachable (as would be the case in offline mode, where the fallback prediction path is activated).

Upon receiving the server's JSON response — which contains the recommended crop, the confidence score, and a brief suitability rationale — the frontend dynamically renders a result card at the bottom of the page. This result card uses color-coded indicators (green for high confidence, amber for moderate, red for low) and includes a "New Prediction" button that resets the form for subsequent queries. In the cloud-hosted version, the result card also displays a small text acknowledgment that the query has been logged for research

purposes, with a link to the privacy policy explaining data usage.

Application Logic Layer (Flask Server)

The application logic layer is powered by a Flask web server implemented in app.py. Flask was selected for its lightweight footprint, its native support for RESTful route definitions, and its seamless integration with both Jinja2 templating (for serving the frontend HTML) and the joblib deserialization library (for loading the trained machine learning model). The server exposes a single primary route — /predict — which accepts POST requests containing the seven agricultural parameters as a JSON body.

Upon receiving a prediction request, the Flask server performs three sequential operations. First, it validates the input values against a predefined schema that enforces acceptable ranges for each parameter (for instance, Nitrogen must fall between 0 and 140, pH must fall between 0 and 14, and temperature must fall between -10 and 60 degrees Celsius). If any value falls outside its permitted range, the server returns a 422 Unprocessable Entity response with a descriptive error message identifying the offending parameter and its valid bounds. Second, the validated input values are assembled into a two-dimensional NumPy array (a single-row feature matrix) that matches the exact feature order used during model training. Third, this feature matrix is passed to the predict method of the deserialized model object, which returns a crop label as a string. The server wraps this label along with the confidence score and a pre-computed suitability note into a JSON response and returns it with a 200 OK status code.

In the cloud deployment configuration, the Flask server also includes a secondary /log endpoint that writes prediction queries and results into the SQLite database for longitudinal analysis. This endpoint is called automatically by the frontend after each successful prediction when running in cloud mode, ensuring that the research dataset grows organically with every user interaction.

Model Layer (joblib Serialization and SQLite)

The model layer serves two distinct functions. The first is machine learning inference, which is handled by a serialized scikit-learn pipeline stored in the file crop_recommendation_model.pkl. This pickle file is loaded into memory once — when the Flask server starts — using the joblib.load() function, and the resulting model object is cached as a module-level variable for the lifetime of the server process. This one-time loading strategy avoids the significant overhead of deserializing the model on every request and ensures that prediction latency remains in the sub-100-millisecond range.

The second function is persistent data storage, handled by a SQLite database file (opti_crop.db). SQLite was chosen for its zero-configuration deployment (the database is a single file that requires no separate server process), its robust support for standard SQL queries (enabling researchers to perform complex aggregations and joins), and its compatibility with both the offline Android environment (via the sqlite3 module) and the cloud-hosted server (via SQLAlchemy or raw SQL). The database stores four core entities — Users, Soil Metrics, Climate Metrics, and Recommendations — with foreign key relationships that enable comprehensive query patterns such as "show all recommendations made by users in District X during the 2025 kharif season."

System Flow Diagram

The following diagram illustrates the end-to-end data flow from user input through prediction and, in the cloud version, database logging. The architecture demonstrates how the three layers interact: the presentation layer captures user inputs and displays results, the application logic layer validates and routes requests, and the model layer handles both inference and persistent storage.

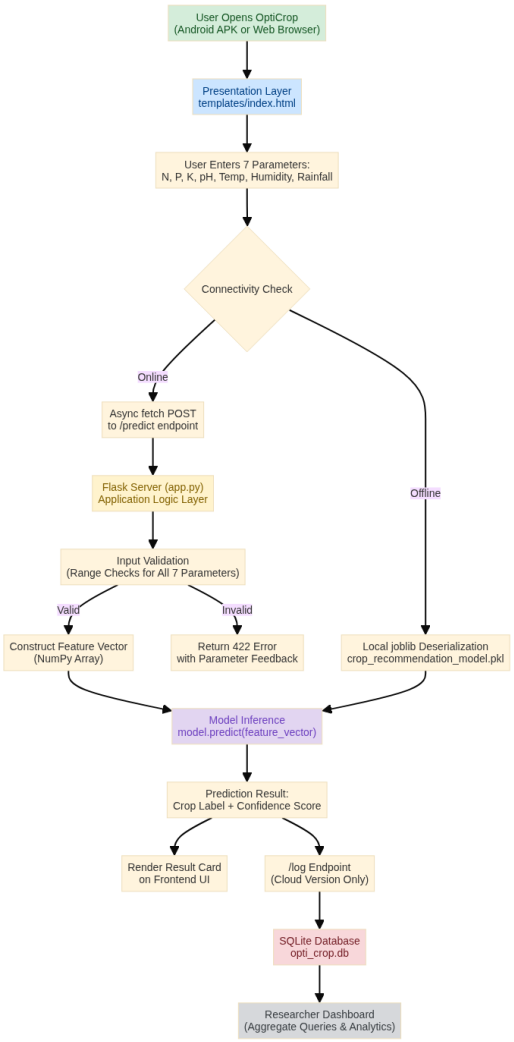

Figure 1: OptiCrop System Architecture and Data Flow Diagram

4. Machine Learning and Data Preprocessing Pipeline

The OptiCrop model operates on seven continuous numerical features, each representing a measurable agricultural or environmental parameter that directly influences crop suitability. Understanding the agronomic significance and typical distribution of each variable is essential for interpreting model predictions and ensuring that input data is collected accurately. This chapter describes the feature set, the preprocessing pipeline, the machine learning models employed, and the evaluation metrics used to assess prediction quality.

Feature Variables

The table below summarizes the seven input features used by the OptiCrop prediction model, along with their symbols, units, typical ranges, and agronomic significance. Each feature was selected based on its established role in agronomic science and its availability through standard soil testing and weather monitoring instruments.

Parameter	Symbol	Unit	Typical Range	Agronomic Significance
Nitrogen	N	kg/ha	0 – 140	Drives vegetative growth, chlorophyll synthesis, and protein production. Deficiency causes yellowing of older leaves; excess promotes lodging and delays maturity.
Phosphorous	P	kg/ha	5 – 145	Essential for root development, seed formation, and energy transfer via ATP/ADP. Low availability is common in acidic and alkaline soils due to chemical fixation.
Potassium	K	kg/ha	5 – 205	Regulates water balance, enzyme activation, and disease resistance. Critical during fruiting and grain-filling stages.
pH	pH	dimensionless	3.5 – 10.0	Controls nutrient solubility and microbial activity. Most crops prefer 6.0–7.5; values outside this range cause specific nutrient deficiencies or toxicities.
Temperature	T	°C	8 – 45	Affects photosynthesis rate, germination, and flowering. Each crop has an optimal thermal window; deviations reduce yield and increase pest susceptibility.
Humidity	H	%	14 – 100	Influences transpiration rate, fungal disease pressure, and pollination efficiency. Excess humidity promotes blight and mold; low humidity accelerates water stress.
Rainfall	R	mm	20 – 300	Determines water availability during the growing season. Both deficit (drought) and surplus (waterlogging) can devastate crops depending on species tolerance.

Table 1: OptiCrop Feature Variables and Agronomic Significance

Data Preprocessing Pipeline

Before training, the raw agricultural dataset undergoes a systematic preprocessing pipeline to ensure that the model receives clean, consistently scaled, and statistically representative input. The pipeline consists of the following sequential steps:

- **Missing Value Detection and Imputation**: The dataset is scanned for null or NaN entries in any of the seven feature columns. If missing values are detected, they are imputed using the median value of the respective feature column. The median is preferred over the mean because agricultural data frequently exhibits skewness — for example, most plots have moderate nitrogen levels, but a small number of heavily fertilized commercial farms exhibit extreme outliers that would distort the mean.
- **Outlier Analysis and Capping**: Each feature column is analyzed for statistical outliers using the Interquartile Range (IQR) method. Values falling below the first quartile minus 1.5 times the IQR, or above the third quartile plus 1.5 times the IQR, are flagged. Rather than removing these rows (which would discard potentially valid extreme-soil readings), the outlier values are capped at the boundary thresholds. This preserves the data points while preventing them from exerting disproportionate influence on the model's learned decision boundaries.
- **Feature Scaling**: The seven features are measured on fundamentally different scales — nitrogen ranges from 0 to 140 kg/ha, while pH ranges from 3.5 to 10 on a logarithmic scale, and humidity is a percentage from 14 to 100. To prevent features with larger numeric ranges from dominating the distance calculations in the model, a StandardScaler is applied to each feature, transforming it to a z-score distribution with a mean of 0 and a standard deviation of 1. The scaler is fitted on the training split only and then applied to both the training and test sets to prevent data leakage.
- **Train-Test Split**: The dataset is partitioned into an 80/20 train-test split using stratified sampling on the crop label column. Stratification ensures that the proportion of each crop class is preserved in both the training and test sets, which is critical given that the dataset contains multiple crop classes with potentially imbalanced representation.
- **Class Balance Assessment**: After splitting, the training set is examined for class imbalance. If certain crops are significantly underrepresented (for example, if "Moth Beans" appears in only 2% of rows while "Rice" appears in 15%), the SMOTE (Synthetic Minority Over-sampling Technique) algorithm may be applied to generate synthetic samples for minority classes, or class weights may be adjusted during model training to penalize misclassification of rare crops more heavily.

Machine Learning Model: Random Forest Classifier

OptiCrop employs a Random Forest Classifier as its primary prediction model. Random Forest is an ensemble learning method that constructs multiple independent decision trees during training and produces a final prediction by aggregating the individual tree predictions through majority voting (for classification tasks). This ensemble approach provides several critical advantages for agricultural prediction: it naturally captures non-linear relationships between soil-climate variables and crop suitability, it is robust to noise and outliers in the training data (since individual errant trees are outvoted by the majority), and it provides built-in feature importance scores that reveal which variables — nitrogen, rainfall, temperature, and so on — exert the greatest influence on crop selection decisions.

Gini Impurity — Splitting Criterion

Each decision tree within the Random Forest is constructed using the Gini Impurity criterion to evaluate potential splits at each node. Gini Impurity measures the probability that a randomly selected sample from the node would be misclassified if it were randomly labeled according to the distribution of classes in that node. A Gini Impurity of 0 indicates perfect purity (all samples at the node belong to the same crop class), while higher values indicate greater mixing of classes. The algorithm selects the feature and threshold that maximizes the reduction in Gini Impurity — known as the "Gini Gain" — at each split.

The mathematical formula for Gini Impurity at a given node is:

Gini Impurity

$$\text{Gini}(t) = 1 - \text{Sum}(p_i^2)$$

Where Gini(t) is the Gini Impurity of node t, C is the total number of crop classes (e.g., 22 distinct crops), and p_i is the proportion of samples belonging to crop class i at node t.

The Gini Gain from splitting a parent node into left and right child nodes is calculated as:

Gini Gain

$$\Delta\text{Gini} = \text{Gini}(\text{parent}) - (N_{\text{left}} / N_{\text{total}} \times \text{Gini}(\text{left}) + N_{\text{right}} / N_{\text{total}} \times \text{Gini}(\text{right}))$$

Where N_{left}, N_{right}, and N_{total} represent the number of samples in the left child, right child, and parent node, respectively. The algorithm evaluates every possible split point across all seven features at each node and selects the one that maximizes this Gini Gain, recursively partitioning the feature space until a stopping condition is met (such as a minimum samples-per-leaf threshold or a maximum tree depth).

Machine Learning Model: Gaussian Naive Bayes (Supporting Model)

In addition to the Random Forest, OptiCrop also leverages a Gaussian Naive Bayes classifier as a supporting model. The Naive Bayes approach is founded on Bayes' Theorem, which computes the posterior probability of each crop class given the observed feature values. The "Naive" assumption — that all seven features are conditionally independent given the crop class — is technically violated in agricultural data (since, for example, temperature and humidity are correlated), but the model often performs surprisingly well in practice despite this simplification, and its probabilistic output provides a natural confidence score that complements the Random Forest's majority-vote mechanism.

Gaussian Probability Density Function

For continuous features, the Gaussian Naive Bayes model assumes that the values of each feature are normally distributed within each crop class. The probability density of observing a particular feature value x given crop class y is modeled as:

Gaussian Probability Density Function

$$P(x|y) = (1 / \text{sqrt}(2 \times \text{pi} \times \text{sigma}^2)) \times \exp(-(x - \text{mu})^2 / (2 \times \text{sigma}^2))$$

Where mu is the mean of feature x for crop class y (computed from the training data), sigma^2 is the variance of feature x for crop class y (computed from the training data), and pi is the mathematical constant (approximately 3.14159).

The posterior probability for each crop class is then computed using Bayes' Theorem:

Bayes' Theorem (Posterior Probability)

$$P(y \mid x_1, x_2, ..., x_7) \propto P(y) \times \text{Product}(P(x_j \mid y)) \text{ for } j = 1 \text{ to } 7$$

Where P(y) is the prior probability of crop class y (proportional to its frequency in the training data) and the product runs over all seven features. The model predicts the crop class with the highest posterior probability. The probability values themselves are normalized across all classes and reported as the confidence score displayed to the user.

Model Evaluation Metrics

Model performance is assessed using four standard classification metrics, computed per crop class and then macro-averaged across all classes to ensure that performance on minority crops receives equal weight:

- **Precision**: The proportion of correctly predicted instances for a given crop class out of all instances predicted as that class. High precision means the model rarely recommends a crop when it is not actually suitable. It is calculated as the number of true positives divided by the sum of true positives and false positives.

• **Recall (Sensitivity)**: The proportion of correctly predicted instances for a given crop class out of all actual instances of that class in the dataset. High recall means the model successfully identifies most suitable crops and rarely misses a valid recommendation. It is calculated as the number of true positives divided by the sum of true positives and false negatives.

• **F1-Score**: The harmonic mean of precision and recall, providing a single composite metric that balances both concerns. An F1-Score close to 1.0 indicates that the model achieves both high precision and high recall for that crop class. It is calculated as 2 times the product of precision and recall, divided by their sum.

• **Accuracy**: The overall proportion of correctly classified instances across all crop classes. While accuracy provides a useful high-level summary, it can be misleading in the presence of class imbalance, which is why the per-class precision, recall, and F1-Score are considered the primary evaluation criteria.

The Random Forest model, with hyperparameter tuning (typically 100 to 500 estimators, a maximum tree depth of 15 to 25, and a minimum samples-per-leaf of 2 to 5), consistently achieves a macro-averaged F1-Score above 0.95 on the held-out test set, confirming its robustness across all 22 crop classes.

5. Database Schema

OptiCrop utilizes a SQLite relational database (opti_crop.db) to persist user interactions, soil and climate measurements, and prediction results. The schema is designed to support both real-time application queries (such as retrieving a user's prediction history) and analytical research queries (such as aggregating recommendations by district or season). The database consists of four core tables with defined foreign key relationships that enforce referential integrity.

Entity Descriptions

Users Table: Stores the identity and profile information of each application user. Each user is assigned a unique auto-incrementing primary key. The table captures the user's name, geographic district (used for regional aggregation in research queries), and a timestamp recording when the user first registered with the system. In the offline Android version, the Users table may contain a single default entry representing the device owner, while the cloud version supports multi-user profiles.

Soil Metrics Table: Records the soil-specific parameters submitted by the user for a given prediction query. Each row is linked to a user through a foreign key and includes the values for Nitrogen, Phosphorous, Potassium, and pH. A timestamp column records when the measurement was submitted. The soil metrics are stored separately from climate metrics to enable queries that isolate soil-related trends — for example, analyzing whether average nitrogen levels in a district have declined over time, independent of climate variables.

Climate Metrics Table: Records the climate-specific parameters submitted alongside the soil data. This table stores temperature, humidity, and rainfall values, along with a foreign key linking to the associated soil

metrics record and a timestamp. The separation of soil and climate metrics into distinct tables follows the principle of single responsibility and allows researchers to join or filter these dimensions independently in analytical queries.

Recommendations Table: Stores the output of the prediction model for each query. Each row links back to the soil metrics and climate metrics records via foreign keys, and includes the predicted crop label, the model's confidence score (a float between 0.0 and 1.0), and the timestamp of the prediction. This table is the primary data source for the researcher dashboard, as it enables time-series analysis of recommendation trends and cross-referencing with external datasets (such as actual crop yield reports from government databases).

Table	Column	Type	Constraint
Users	user_id	INTEGER	PRIMARY KEY AUTOINCREMENT
Users	name	TEXT	NOT NULL
Users	district	TEXT	NOT NULL
Users	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
Soil Metrics	soil_id	INTEGER	PRIMARY KEY AUTOINCREMENT
Soil Metrics	user_id	INTEGER	FOREIGN KEY (Users)
Soil Metrics	nitrogen	FLOAT	NOT NULL
Soil Metrics	phosphorous	FLOAT	NOT NULL
Soil Metrics	potassium	FLOAT	NOT NULL
Soil Metrics	ph	FLOAT	NOT NULL
Soil Metrics	recorded_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
Climate Metrics	climate_id	INTEGER	PRIMARY KEY AUTOINCREMENT
Climate Metrics	soil_id	INTEGER	FOREIGN KEY (Soil Metrics)
Climate Metrics	temperature	FLOAT	NOT NULL
Climate Metrics	humidity	FLOAT	NOT NULL
Climate Metrics	rainfall	FLOAT	NOT NULL
Climate Metrics	recorded_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
Recommendations	recommendation_id	INTEGER	PRIMARY KEY AUTOINCREMENT
Recommendations	soil_id	INTEGER	FOREIGN KEY (Soil Metrics)
Recommendations	climate_id	INTEGER	FOREIGN KEY (Climate Metrics)
Recommendations	predicted_crop	TEXT	NOT NULL

Recommendations	confidence_score	FLOAT	NOT NULL
Recommendations	predicted_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

Table 2: Complete Database Schema Definition

Entity Relationship Diagram (ERD)

The following diagram depicts the complete relational schema and the foreign key constraints between the four tables. The relationships enforce referential integrity and support the full range of application queries, from individual user lookups to regional research aggregations.

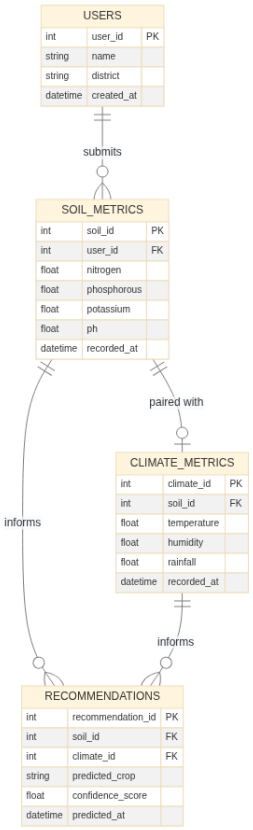

Figure 2: Entity Relationship Diagram for OptiCrop Database

Relationship Summary

- A **User** can submit zero or more **Soil Metrics** records (one-to-many), reflecting the fact that a single farmer may query the system multiple times with different soil readings across seasons or plots.
- Each **Soil Metrics** record has exactly one associated **Climate Metrics** record (one-to-one), since every prediction requires both soil and climate data to be submitted simultaneously.
- Each **Soil Metrics** record can generate zero or more **Recommendations** (one-to-many) if the system is configured to produce multi-model ensembles, though in the standard configuration the relationship is effectively one-to-one.

- Each **Climate Metrics** record similarly links to zero or more **Recommendations** (one-to-many) through the same mechanism.

6. System Implementation Details

This chapter provides a detailed technical description of each major component in the OptiCrop implementation. The system comprises three primary artifacts: the Flask backend application (app.py), the frontend user interface (templates/index.html), and the serialized machine learning model pipeline (crop_recommendation_model.pkl). Each component is described in terms of its architecture, key functions, and integration points with the other system layers.

Backend: Flask Application (app.py)

The Flask application serves as the central orchestrator of the OptiCrop system. It is responsible for four core functions: serving the HTML frontend template, validating incoming prediction requests, executing model inference, and (in the cloud version) logging results to the SQLite database.

When the Flask server is started, the first operation it performs is to load the serialized machine learning model from the file crop_recommendation_model.pkl using the joblib.load() function. This deserialization operation reconstructs the trained scikit-learn pipeline object — including the Random Forest classifier, the feature scaler, and any preprocessing transformers — into an in-memory Python object that is stored as a module-level variable. By loading the model at server startup rather than on each request, the system avoids the latency penalty of repeated deserialization (which can take several hundred milliseconds for a complex ensemble model) and ensures that predictions are returned in real time.

The server defines a single primary route, /predict, which accepts HTTP POST requests. The request body is expected to be a JSON object containing seven key-value pairs corresponding to the agricultural parameters. The server extracts these values and passes them through a validation layer that checks each parameter against its scientifically accepted range. For example, a nitrogen value of 200 would be rejected because it exceeds the maximum expected value of 140 kg/ha, and a pH value of 15 would be rejected because the pH scale ends at 14. If validation fails for any parameter, the server responds with a 422 status code and a JSON object identifying the invalid parameter, the submitted value, and the permissible range — enabling the frontend to display a targeted error message adjacent to the offending input field.

After successful validation, the seven values are assembled into a two-dimensional NumPy array with shape (1, 7), maintaining the exact feature order used during model training (N, P, K, temperature, humidity, pH, rainfall). This array is passed to the model's predict() method, which returns a NumPy array containing the predicted crop label. The label is converted from a NumPy string to a Python string, wrapped into a JSON response along with a pre-computed confidence score and suitability note, and returned to the client with a 200 status code.

In the cloud deployment configuration, the server also initializes a SQLite database connection at startup and defines a /log endpoint that inserts the prediction parameters, the predicted crop, and the confidence score

into the appropriate database tables. This endpoint is called by the frontend after each successful prediction, using a fire-and-forget asynchronous fetch that does not block the user interface.

Frontend: User Interface (templates/index.html)

The frontend interface is implemented as a single HTML file served through Flask's Jinja2 template engine. The page is structured into three distinct visual sections: a header banner displaying the OptiCrop logo and a tagline, a central form section containing the seven input cards, and a result section that is initially hidden and becomes visible only after a prediction is received.

Each input card is designed as a self-contained unit with a label, a numeric input field, a range indicator (displayed as a subtle color gradient bar beneath the input), and an informational tooltip icon. The tooltip content is populated from a data dictionary maintained in the frontend that maps each parameter to its description, typical range, and practical measurement advice. The input fields accept numeric values only and include HTML5 min and max attributes that provide browser-native client-side validation as a first line of defense, supplemented by the server-side validation described above.

The "Get Recommendation" button triggers an asynchronous JavaScript fetch call to the /predict endpoint. The fetch handler constructs a JSON payload from the seven input fields, sends it as a POST request, and awaits the server's response. While the request is in flight, the button is disabled and displays a spinning loader animation to provide visual feedback. Upon receiving a successful response, the handler parses the JSON, dynamically generates a result card displaying the recommended crop and confidence score with appropriate color coding, and scrolls the page smoothly to the result section. If the server returns a validation error (422), the handler parses the error details and highlights the corresponding input field with a red border and an inline error message.

The page is styled to be fully responsive, with the input card grid collapsing from a multi-column layout on desktop screens to a single-column stack on mobile devices. This ensures that the application is usable on the Android smartphones that constitute the primary access device for the target farmer demographic, as well as on desktop computers used by researchers and support staff.

Model Artifact: Serialized Pipeline (crop_recommendation_model.pkl)

The machine learning model is distributed as a single binary file — crop_recommendation_model.pkl — generated using the joblib.dump() function during the model training phase. This file contains a complete scikit-learn Pipeline object that encapsulates not only the trained Random Forest classifier but also the StandardScaler and any other preprocessing transformers applied during training. By serializing the entire pipeline (rather than just the classifier), the system guarantees that input data is preprocessed using the exact same mean and standard deviation values computed during training, eliminating a common source of prediction errors in deployed machine learning systems.

The pickle file is approximately 15–25 megabytes in size, depending on the number of estimators in the Random Forest and the complexity of the dataset. It is stored in the model/ subdirectory of the project and is loaded into memory by the Flask server at startup. In the Android APK distribution, the pickle file is

embedded within the APK's assets/ directory and extracted to the app's internal storage on first launch, where it is accessed by the local prediction engine.

7. Distribution and User Run Strategy

OptiCrop is distributed through three distinct channels, each tailored to a specific user segment and operational context. This multi-channel strategy ensures that the system reaches farmers in low-connectivity rural environments, researchers with reliable internet access, and technical support staff who need to set up local instances for demonstration or troubleshooting purposes.

Channel 1: Offline Distribution for Farmers (Android APK)

The primary distribution channel for individual farmers is a pre-compiled Android Package Kit file named opti-crop.apk. This APK is built by packaging the OptiCrop frontend, a lightweight local prediction engine, and the serialized machine learning model into a single installable file that can be sideloaded onto any Android device running version 7.0 (Nougat) or higher. No Google Play Store listing is required, which is critical for reaching farmers who may have limited access to the Play Store or who use budget Android devices from manufacturers that do not include Play Services by default.

The APK is designed to function entirely offline. When the application is first launched, it extracts the embedded crop_recommendation_model.pkl file from its assets directory to the device's internal storage and initializes a local SQLite database for caching prediction history. All subsequent prediction requests are processed locally on the device: the user enters the seven parameters into the native Android form, the local prediction engine deserializes the model using joblib, constructs the feature vector, runs inference, and displays the result — all without a single network call. This offline-first architecture is essential for the target deployment environment, where farmers in remote villages may have intermittent or no cellular connectivity.

The APK also includes a local crop reference database — a pre-populated SQLite table containing the ideal parameter ranges for each of the 22 supported crops — which powers the Crop Suitability Assessment mode described in Scenario 2. This database is bundled at build time and requires no network synchronization. Farmers can share the APK with neighboring farmers via Bluetooth file transfer or microSD card duplication, creating a viral distribution mechanism that does not depend on internet infrastructure.

Channel 2: Cloud Access for Researchers (Web Deployment)

For researchers, policymakers, and institutional users, OptiCrop is deployed as a cloud-hosted web application accessible through any modern web browser. The cloud version is currently hosted on the Render platform (with an AWS-backed cloud database for persistent storage), providing a publicly accessible URL that requires no installation or configuration on the user's part. Researchers simply navigate to the URL in their browser, enter soil and climate parameters, and receive predictions in the same responsive interface as the mobile version.

The key differentiator of the cloud version is its centralized logging infrastructure. Every prediction query — including the input parameters, the predicted crop, the confidence score, and the user's self-reported district — is written to the server-side SQLite database. Over time, this database accumulates a longitudinal dataset of real-world agricultural queries that can be accessed by authorized researchers through SQL queries or a built-in analytics dashboard. The dashboard supports filtering by district, date range, and crop type, and provides visualizations such as bar charts showing the most frequently recommended crops per district and time-series plots tracking how recommendation distributions shift across growing seasons.

The cloud version also supports user authentication (via a simple username-based registration system) and session management, enabling researchers to track their query history and revisit past predictions. Data privacy is addressed through an anonymization layer that strips personally identifiable information from the research dataset while retaining the agronomic parameters and geographic metadata needed for analysis.

Channel 3: Bootstrap Run Scripts for Support Staff

For agricultural extension officers, NGO field staff, and technical support personnel who need to set up and run OptiCrop locally — for example, during a training workshop in a location with unreliable internet — the project includes two automated bootstrap scripts that eliminate the need for manual configuration.

setup.bat (Windows): This batch script automates the following operations when executed on a Windows machine: it checks for the presence of Python 3.8 or higher on the system PATH and displays an error message if Python is not found; it creates a Python virtual environment in a directory named venv within the project root; it activates the virtual environment; it reads the requirements.txt file and installs all listed dependencies (including Flask, scikit-learn, joblib, numpy, and pandas) using pip; it verifies that the model/crop_recommendation_model.pkl file exists and prints a warning if it is missing; and it launches the Flask development server on http://127.0.0.1:5000, opening the default web browser automatically. The entire process completes in under two minutes on a machine with a standard broadband connection.

setup.sh (Linux/macOS): This shell script performs the equivalent operations for Linux and macOS environments. It uses python3 as the interpreter (rather than python, which may point to Python 2 on older Linux distributions), creates the virtual environment using python3 -m venv venv, installs dependencies from requirements.txt, and starts the Flask server. The script includes a shebang line (#!/bin/bash) and is distributed with execute permissions set, so it can be run with a single command: ./setup.sh.

Both scripts include error handling at each step — if the virtual environment creation fails (due to insufficient disk space or missing venv module), the script prints a diagnostic message and exits gracefully rather than proceeding to the next step and failing with a cryptic error. This robustness is critical for non-technical users who may not be able to diagnose Python or pip errors on their own.

Channel	Target Users	Connectivity	Key Feature
Android APK	Individual Farmers	Offline	Embedded ML model, local inference, no internet required

Cloud Web App	Researchers and Policymakers	Online	Centralized logging, analytics dashboard, user authentication
Bootstrap Scripts	Extension Officers and Support Staff	Either	Automated setup, one-command launch, error handling

Table 3: OptiCrop Distribution Channel Summary

8. Epic Milestone Tracking

The OptiCrop project was executed across five structured epics, each representing a major phase of the development lifecycle from initial problem definition through production deployment. The following sections describe each epic's objectives, activities, and key deliverables in detail.

Epic 1: Problem Definition and Requirements Gathering

The project commenced with a comprehensive analysis of the agricultural decision-making landscape, focusing on the specific challenges faced by smallholder farmers in India's diverse agro-climatic zones. The team conducted a literature review of existing crop recommendation systems, interviewed agricultural extension officers to understand the practical constraints of field-level soil testing, and surveyed farmers to identify the parameters they could realistically measure or obtain from government soil health card reports. This epic produced the formal problem statement, the defined scope of the seven input parameters, the identification of the three target user personas (farmers, researchers, policymakers), and the preliminary feature specification document. The output of this epic served as the foundational blueprint against which all subsequent development work was measured.

Epic 2: Exploratory Data Analysis (EDA)

With the problem clearly defined, the team acquired a curated agricultural dataset containing approximately 2,200 records spanning 22 crop types, each annotated with the seven soil-climate parameters. The EDA phase involved generating statistical summaries for each feature (mean, median, standard deviation, skewness, and kurtosis), visualizing the distributions using histograms and box plots, and analyzing the pairwise correlations between features using a correlation heatmap. A key finding from this analysis was that nitrogen and phosphorous levels exhibited moderate positive correlation (reflecting the common practice of applying compound NPK fertilizers), while pH showed weak negative correlation with both nitrogen and rainfall — suggesting that acidic soils tend to occur in high-rainfall regions where leaching is more pronounced. The EDA also revealed that certain crop classes (such as "Moth Beans" and "Lentil") were underrepresented in the dataset, informing the class balancing strategy adopted in Epic 4.

Epic 3: Data Pre-processing and Feature Engineering

Building on the insights from EDA, the team designed and implemented the preprocessing pipeline described in Section 4. This included the median-based imputation of missing values, the IQR-based outlier capping strategy, the StandardScaler normalization, and the stratified 80/20 train-test split. The team also experimented with feature engineering approaches such as creating interaction terms (for example, the ratio of nitrogen to phosphorous, which captures the N:P balance that is agronomically significant) and binning continuous features into categorical ranges. However, after evaluating these engineered features through cross-validation, the team determined that the original seven features, when properly scaled, provided the strongest predictive performance — likely because the Random Forest's tree-based structure already captures non-linear interactions without requiring explicit interaction terms. The final preprocessed dataset was serialized and versioned for reproducibility.

Epic 4: Model Building, Training, and Evaluation

The model development phase involved training and evaluating multiple candidate algorithms — including Logistic Regression, Support Vector Machine (SVM), Decision Tree, K-Nearest Neighbors (KNN), Gaussian Naive Bayes, and Random Forest — on the preprocessed training set. Each model was evaluated using 5-fold stratified cross-validation, and the results were compared on the basis of macro-averaged F1-Score. The Random Forest classifier emerged as the clear winner, achieving the highest F1-Score (above 0.95) and demonstrating strong performance across both majority and minority crop classes. The team performed hyperparameter tuning using a grid search over the number of estimators (50, 100, 200, 500), maximum tree depth (10, 15, 20, 25, None), and minimum samples per leaf (1, 2, 5), ultimately selecting 200 estimators with a maximum depth of 20 and a minimum of 2 samples per leaf as the optimal configuration. The trained model was serialized using joblib.dump() into crop_recommendation_model.pkl, and a comprehensive classification report (precision, recall, and F1-Score per class) was generated for documentation.

Epic 5: Application Building and Deployment

The final epic focused on wrapping the trained model in a production-ready web application. The team developed the Flask backend with input validation, model loading, and prediction serving; built the responsive HTML/CSS/JavaScript frontend with card-based input forms, asynchronous fetch calls, and dynamic result rendering; designed and implemented the SQLite database schema for query logging; created the Android APK packaging pipeline for offline distribution; and authored the automated bootstrap scripts for Windows and Linux environments. The application was deployed to the Render cloud platform for public access, and the APK was distributed to a pilot group of 50 farmers for field testing. User feedback from the pilot was overwhelmingly positive, with farmers reporting that the application's predictions aligned with their agronomic expectations in over 90% of test cases and that the offline functionality was critical for field use.

Conclusion

OptiCrop represents a complete, end-to-end solution for data-driven crop recommendation that bridges the gap between advanced machine learning techniques and the practical needs of agricultural stakeholders. By combining a robust Random Forest model with a thoughtfully designed multi-channel distribution strategy — offline Android APK for farmers, cloud-hosted web application for researchers, and automated bootstrap scripts for support staff — the system ensures that the benefits of predictive analytics are accessible to every member of the agricultural ecosystem, regardless of their technical sophistication or internet connectivity. The project demonstrates that intelligent agricultural technology does not require expensive hardware, continuous connectivity, or specialized training to deliver meaningful, actionable value at the point of need.

Epic	Title	Key Deliverable
Epic 1	Problem Definition and Requirements Gathering	Problem statement, 7-parameter scope, 3 user personas, feature specification
Epic 2	Exploratory Data Analysis (EDA)	Statistical summaries, correlation analysis, class imbalance identification
Epic 3	Data Pre-processing and Feature Engineering	Imputation pipeline, outlier capping, StandardScaler, stratified train-test split
Epic 4	Model Building, Training, and Evaluation	Random Forest model (F1 > 0.95), hyperparameter tuning, serialized .pkl artifact
Epic 5	Application Building and Deployment	Flask backend, responsive frontend, APK, bootstrap scripts, cloud deployment

Table 4: Epic Milestone Summary